



EDITORA AGÊNTICA

DO SDLC CLÁSSICO AO AI-FIRST: POR QUE O CONTRATO MUDOU

Uma leitura prática sobre o desenvolvimento orientado a agentes

Heverton Eduardo Peres

ESPECIALISTA EM MARKETING E DESENVOLVIMENTO DE SOLUÇÕES

Heverton Eduardo Peres

Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou	5
Capítulo 1: Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou	6
Introdução	6
Explica	6
Ilustra	7
Técnica	8
O Contrato de Fase como Artefato Técnico	8
A Matriz RACI em Código	9
Medindo o Custo de Contexto	10
O Motor de Autorização com Gate de Artefato	11
O Contrato de Transição como Código de Equipe	13
O Custo do Contexto como Dado de Projeto	13
A Fronteira entre Especificação e Invenção	13
O Mapa de Fases com Entradas e Saídas	14
O Estudo de Caso: Duas Organizações, Dois Destinos	15
O Modelo de Decisão de Adoção	15
O Modelo de Gap de Fases com Custo de Contexto	16
O Modelo de Comparação entre Ciclos	16
O Modelo de Avaliação de Prontidão do Time	17
O Modelo de Análise de Gap do Ciclo	18
O Modelo de Estimativa de Adoção	19
O Modelo de Análise de Gap do Ciclo	20
O Monitor de Fases em Tempo Real	20
O Modelo de Gate com Critérios Pesados	22
A Esteira de Validação de Transição	22
O Modelo de Registro de Fase com Evidência	23
O Diagnóstico do Ciclo Atual em Cinco Perguntas	24
O Exemplo do Mundo Real: a Fábrica Agêntica	24
A Economia da Mudança: o Cálculo do Retorno	25
O Manifesto do Ciclo	25
Passos Numerados para Adotar o Modelo na Sua Equipe	26
Aplica	26
Conclusão	27
Por que este capítulo importa	27

Conceitos-chave deste capítulo	28
Checklist para aplicar hoje	28
Perguntas que você deve se fazer	28
Glossário rápido	29
O erro mais comum nesta fase	29
Um exemplo concreto para fixar	29
Próximos Passos	30

Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou

Uma leitura direta e prática para quem quer levar o desenvolvimento orientado a agentes a sério — sem jargão acadêmico, com exemplos aplicáveis.

Capítulo 1: Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou

Introdução

Bem-vindo ao primeiro voo da sua jornada como Comandante de Operações de Software. Este capítulo responde à pergunta mais fundamental da obra inteira: por que o ciclo de vida de desenvolvimento de software — aquele processo disciplinado de fases que sustenta a indústria há décadas — está sendo redesenhado na era dos agentes de IA? Você vai aprender a diferença estrutural entre o SDLC clássico e o SDLC AI-first, entender por que o artefato-mestre deixou de ser o documento para ser a spec executável mais testes, e reconhecer a mudança de custo dominante: de horas-homem para tokens e contexto.

Ao final, você será capaz de explicar para qualquer equipe — sem jargão — por que “pedir código para uma IA” é só a ponta de um iceberg cujo nome real é governança de ciclo de vida.

Explica

O Software Development Life Cycle, ou ciclo de vida de desenvolvimento de software, é o modelo organizacional que estrutura o trabalho de construção de software em fases: levantamento de requisitos, design, implementação, testes, implantação e manutenção. Cada modelo clássico — Waterfall, Espiral, RUP, Ágil, Scrum — é, no fundo, uma resposta diferente à mesma pergunta: como coordenar trabalho intelectual caro e propenso a erro.

A premissa silenciosa de todos esses modelos é que **o executante é humano**. Requisitos são escritos por analistas, design por arquitetos, código por programadores e testes por QA. O SDLC clássico otimiza, portanto, fases manuais: distribui horas-homem, cria artefatos intermediários para comunicação entre especialistas e usa a revisão humana como guarda de qualidade.

O paradigma AI-first quebra essa premissa. O agente de IA não é uma ferramenta de autocomplete glorificada — é um **executante autônomo** que planeja, escreve, testa e corrige

código com loop de execução próprio. Estudos acadêmicos mostram que a geração isolada de trechos via LLM está dando lugar a agentes orientados a ferramentas que cobrem requisitos, geração de código, decisão autônoma, design, testes e manutenção. A pergunta deixa de ser “como o humano usa a IA para digitar mais rápido” e passa a ser “como o humano governa um executante que trabalha sozinho”.

É aqui que a metáfora da torre de controle de tráfego aéreo se torna operacional. Um avião moderno voa sozinho a maior parte do tempo — o piloto automático gerencia altitude, velocidade e rota. Mas ninguém em sã consciência tira do controlador de voo a autoridade de decolar, desviar e pousar. O piloto automático é o agente; o controlador de voo é o humano. No SDLC AI-first, cada fase é um voo que decola apenas com plano aprovado (a spec), é monitorado em tempo real (observabilidade) e só aterriza com verificação adversarial.

A mudança de custo dominante é igualmente radical. No SDLC clássico, o recurso escasso é o tempo humano. No AI-first, o recurso escasso é o **contexto**: tokens, janelas de contexto, rate limits e vida útil de sessão. Uma decisão de arquitetura que custaria uma hora de reunião agora custa milhares de tokens; uma sessão de agente que esgota o contexto no meio do build é tão cara quanto um programador que desiste no meio da feature. O design do ciclo de vida precisa, portanto, tratar a economia de contexto como variável de primeira classe, não como detalhe de infraestrutura.

A literatura de 2025-2026 converge nessa direção. O relatório DORA de 2025, em sua agenda inaugural sobre desenvolvimento assistido por IA, encontrou correlação positiva entre adoção de IA e throughput de entrega (frequência de deploy e lead time), mas alerta que a estabilidade — a taxa de mudanças que causam falha — pode piorar quando a governança não acompanha. Em outras palavras: IA sem ciclo de vida disciplinado é dívida técnica com pedal no acelerador.

Há também a dimensão ética e social. Pesquisadores defendem que o agêntico deve ser expandido para o “processo inteiro” — requisitos, arquitetura, desenvolvimento e operações — com alinhamento ético desde a origem. O SDLC AI-first não é sobre máquinas que escrevem código; é sobre humanos que autorizam, arbitram e evoluem contratos em um ambiente onde a execução foi delegada.

Ilustra

Imagine o aeroporto de Congonhas em horário de pico. Dezenas de aeronaves precisam decolar e pousar em pistas compartilhadas, com margens de segurança de segundos. Nenhum piloto decide sozinho o momento da decolagem — há uma torre de controle que autoriza cada movimento. O piloto tem autonomia dentro do voo, mas a decolagem exige plano de voo aprovado, a rota é monitorada pelo radar, e o pouso exige autorização.

Agora transporte essa imagem para o software. O SDLC clássico é o aeroporto sem torre: cada equipe decola quando quer, com planos de voo que são documentos que ninguém lê, e o radar é uma reunião mensal de status. O SDLC AI-first é o mesmo aeroporto com uma torre moderna: o piloto automático (agente) é poderoso, mas o controlador (humano) mantém a autoridade sobre cada transição de fase.

[Diagrama do capítulo omitido neste formato.]

Como Comandante de Operações de Software, você já percebe aqui o primeiro reflexo profissional: **nenhuma decolagem sem plano aprovado**. O piloto automático não decide quando decolar — ele obedece à torre. Da mesma forma, o agente não decide quando iniciar uma implementação sem uma spec aprovada. Essa é a essência do spec-driven development, que aprofundaremos no Capítulo 3.

Técnica

O Contrato de Fase como Artefato Técnico

A transição do SDLC clássico para o AI-first não é uma decisão filosófica — é uma decisão de engenharia com artefatos concretos. O primeiro passo técnico é transformar o ciclo de vida em algo **inspecionável por máquina**. Vamos construir uma representação declarativa do ciclo de vida em JSON que uma esteira agêntica consegue interpretar.

```
{
  "nome_ciclo": "SDLC AI-first",
  "versao": "1.0",
  "fases": [
    {
      "id": "intencao", "entrada": "problema", "saida":
      "intencao_1_paragrafo", "humano_responsable": true,
      "agente_pode_implementar": false},
    {
      "id": "spec", "entrada": "intencao", "saida": "spec_executavel",
      "humano_responsable": false, "agente_pode_implementar": false},
    {
      "id": "design", "entrada": "spec_aprovada", "saida":
      "contratos_de_modulos", "humano_responsable": true,
      "agente_pode_implementar": false},
    {
      "id": "build", "entrada": "spec_e_design", "saida": "codigo_e_testes",
      "humano_responsable": false, "agente_pode_implementar": true},
    {
      "id": "verificar", "entrada": "diff_verde", "saida":
      "parecer_adversarial", "humano_responsable": true,
      "agente_pode_implementar": false},
    {
      "id": "entregar", "entrada": "merge_aprovado", "saida": "release",
```



```

"humano_responsable": true, "agente_pode_implementar": false},
  {"id": "operar", "entrada": "release", "saida": "metricas_e_logs",
"humano_responsable": false, "agente_pode_implementar": false},
  {"id": "evoluir", "entrada": "observacoes", "saida":
"skills_e_specs_revisadas", "humano_responsable": true,
"agente_pode_implementar": false}
],
"regra_ouro": "nenhuma_fase_avanca_sem_artefato_de_saida_validado"
}

```

A Matriz RACI em Código

A distribuição de responsabilidade entre humano e agente — que no SDLC clássico é um documento PowerPoint — vira uma estrutura de decisão programável. O trecho abaixo define, em Python, um motor de autorização de fase: o agente só avança quando o contrato da fase anterior está satisfeito.

```

from dataclasses import dataclass, field
from enum import Enum
from typing import List, Dict

class Papel(Enum):
    HUMANO_RESPONSABLE = "humano_responsable"
    AGENTE_RESPONSABLE = "agente_responsable"
    HUMANO_ACCOUNTABLE = "humano_accountable"

@dataclass
class Fase:
    id: str
    entrada: str
    saida: str
    humano_decide: bool
    agente_executa: bool
    artefatos_obrigatorios: List[str] = field(default_factory=list)

    def pode_avancar(self, artefatos: Dict[str, bool]) -> tuple:
        faltando = [a for a in self.artefatos_obrigatorios if not
artefatos.get(a, False)]
        if faltando:
            return False, f"artefatos pendentes: {'', '.join(faltando)}"
        return True, "contrato de fase satisfeito"

```

```

FASES = {
    "intencao": Fase("intencao", "problema", "intencao_1_paragrafo", True,
False),
    "spec": Fase("spec", "intencao", "spec_executavel", True, False),
    "design": Fase("design", "spec_aprovada", "contratos_de_modulos", True,
False),
    "build": Fase("build", "spec_e_design", "codigo_e_testes", False,
True),
    "verificar": Fase("verificar", "diff_verde", "parecer_adversarial",
True, False),
    "entregar": Fase("entregar", "merge_aprovado", "release", True, False),
    "operar": Fase("operar", "release", "metricas_e_logs", False, False),
    "evoluir": Fase("evoluir", "observacoes", "skills_e_specs", True,
False),
}

def autorizar_avancar(fase_id: str, artefatos: Dict[str, bool]) -> None:
    fase = FASES[fase_id]
    ok, motivo = fase.pode_avancar(artefatos)
    status = "DECOLAGEM AUTORIZADA" if ok else "DECOLAGEM BLOQUEADA"
    print(f"[{status}] {fase_id}: {motivo}")
    if not ok:
        raise SystemExit(1)

if __name__ == "__main__":
    # Simulacao: spec ainda nao aprovada -> build nao decola
    artefatos = {
        "intencao_1_paragrafo": True,
        "spec_executavel": False,
        "contratos_de_modulos": False,
        "codigo_e_testes": False,
    }
    autorizar_avancar("build", artefatos)

```

Este código é deliberadamente pequeno, mas carrega a tese do capítulo: **a autoridade de transição de fase é código, não costume**. Quando o artefato obrigatório não existe, a decolagem é bloqueada — exatamente como um controlador de voo recusa decolagem sem plano de voo.

Medindo o Custo de Contexto

A segunda mudança de engenharia é tornar o custo do contexto mensurável. Se tokens são o novo recurso escasso, cada fase deve registrar seu consumo. O trecho abaixo instrumenta a sessão do agente.

```

import time
from dataclasses import dataclass

@dataclass
class ConsumoSessao:
    fase: str
    tokens_entrada: int = 0
    tokens_saida: int = 0
    inicio: float = field(default_factory=time.time)

    def registrar(self, entrada: int, saida: int) -> None:
        self.tokens_entrada += entrada
        self.tokens_saida += saida

    def total(self) -> int:
        return self.tokens_entrada + self.tokens_saida

    def resumo(self) -> str:
        duracao = time.time() - self.inicio
        return (f"fase={self.fase} tokens={self.total()} "
                f"duracao={duracao:.0f}s")

sessao_build = ConsumoSessao("build")
sessao_build.registrar(entrada=24_500, saida=3_200)
print(sessao_build.resumo())

```

A ideia é que a esteira colete esses registros na Fase 7 (Operar) e os transforme em insumo da Fase 8 (Evoluir) — por exemplo, uma spec que exigiu 25 mil tokens de entrada para ser redigida pode ser simplificada para 8 mil sem perder escopo.

O Motor de Autorização com Gate de Artefato

A versão produtiva do motor de autorização exige um gate de artefato: além de checar quem pode avançar, ele verifica se o artefato de saída da fase anterior existe e é válido. O código abaixo estende a simulação anterior com o gate — a diferença entre autorizar por papel e autorizar por evidência.

```

import json
from pathlib import Path
from dataclasses import dataclass
from typing import Dict, Optional

```

```

@dataclass
class GateArtefato:
    fase: str
    artefato_obrigatorio: str
    validador: Optional[str] = None

    def verificar(self, raiz: Path) -> tuple:
        caminho = raiz / self.artefato_obrigatorio
        if not caminho.exists():
            return False, f"artefato {self.artefato_obrigatorio}
inexistente"
        if self.validador == "json":
            try:
                json.loads(caminho.read_text(encoding="utf-8"))
            except ValueError as exc:
                return False, f"artefato {self.artefato_obrigatorio}
invalido: {exc}"
            return True, f"gate {self.fase} liberado"

GATES = [
    GateArtefato("spec", "spec_executavel.json", validador="json"),
    GateArtefato("design", "contratos/modulos.json", validador="json"),
    GateArtefato("build", "codigo_e_testes/resultado.txt"),
]

def liberar_fase(fase_id: str, raiz: Path) -> None:
    for gate in GATES:
        if gate.fase == fase_id:
            ok, motivo = gate.verificar(raiz)
            print(f"[{'LIBERADO' if ok else 'BLOQUEADO'}] {gate.fase}:
{motivo}")
            if not ok:
                raise SystemExit(1)

if __name__ == "__main__":
    liberar_fase("spec", Path("exemplos"))

```

A lição técnica: o gate por artefato é mais forte que o gate por papel. Mesmo com o papel correto, a fase não avança sem o artefato válido — o plano de voo existe e foi conferido pela torre.

O Contrato de Transição como Código de Equipe

O SDLC AI-first também muda a forma como o time coordena. A transição de fase deixa de ser discutida em reunião e passa a ser um contrato versionado — uma convenção de equipe que qualquer agente novo lê antes de atuar. O arquivo abaixo é um exemplo de convenção de transição:

```
# Convenção de Transição de Fase – Equipe Plataforma

## Regra geral
Nenhuma fase avança sem o artefato de saída validado (gate de evidência).

## Fases e artefatos
| Fase | Artefato de saída | Validação |
|-----|-----|-----|
| Intenção | intencao.md (1 parágrafo) | humano lê e aprova |
| Spec | spec_executavel.json | JSON válido + 1 critério por requisito |
| Design | contratos/modulos.json | interfaces declaradas |
| Build | codigo_e_testes/ | CI verde (typecheck + testes) |
| Verificar | parecer.md | 3 camadas com evidência |

## Custo de contexto por fase
| Fase | Orçamento de tokens | Responsável |
|-----|-----|-----|
| Spec | 12.000 | analista |
| Build | 60.000 | agente + revisor |
| Verificar | 8.000 | revisor adversarial |
```

Essa convenção é o contrato de transição em formato legível por humanos e por agentes — a ponte entre o mundo dos processos e o mundo do código.

O Custo do Contexto como Dado de Projeto

Quando a economia de tokens se torna variável de projeto, o orçamento do ciclo passa a ser tão explícito quanto o orçamento de horas. Organizações maduras registram, por fase, o consumo de contexto e usam esse dado para calibrar a próxima iteração — uma prática que os consórcios europeus de pesquisa já apontam como diferencial competitivo da adoção de IA no ciclo de vida. Na prática, isso significa que a decisão de delegar uma tarefa ao agente deixa de ser “a IA faz mais rápido” e passa a ser “a IA faz mais rápido E dentro do orçamento de tokens alocado”.

A Fronteira entre Especificação e Invenção

A pesquisa em engenharia de software com LLMs documenta um fenômeno que todo Comandante precisa conhecer: quando a especificação é ambígua, o agente não pergunta — ele inventa.

O estudo de Jin et al. classificou 139 artigos sobre agentes e encontrou a lacuna de requisitos como uma das maiores fontes de defeito estrutural em sistemas agênticos. A especificação executável, que você dominará no Capítulo 3, é exatamente o instrumento que fecha essa lacuna: cada requisito com critério de aceite testável, sem espaço para invenção. Quanto mais cedo o contrato é fechado, menos tokens são gastos em retrabalho — a regra econômica mais rentável do ciclo inteiro.

O Mapa de Fases com Entradas e Saídas

O ponto de partida operacional do AI-first é o mapa explícito do ciclo: cada fase com entrada, saída, executor e critério de avanço. O YAML abaixo é o modelo de mapa que uma equipe preenche uma vez e versiona — a fonte de verdade do processo:

```
ciclo_de_vida:
  nome: "plataforma-pagamentos"
  versao: "1.0"
  fases:
    - id: intencao
      entrada: "problema de negocio"
      saida: "intencao.md (1 paragrafo + DoD)"
      executor: "humano"
      avanco: "intencao aprovada pelo dono do produto"
    - id: spec
      entrada: "intencao aprovada"
      saida: "spec_executavel.json"
      executor: "analista + agente"
      avanco: "todos os requisitos com criterio de aceite"
    - id: design
      entrada: "spec aprovada"
      saida: "contratos/modulos.json + ADRs"
      executor: "arquiteto + agente"
      avanco: "interfaces declaradas e validadas"
    - id: build
      entrada: "design aprovado"
      saida: "codigo + testes + CI verde"
      executor: "agente (test-first)"
      avanco: "suíte completa verde"
    - id: verificar
      entrada: "diff do build"
      saida: "parecer_adversarial.md"
      executor: "revisor independente"
      avanco: "3 camadas com evidência"
    - id: entregar
      entrada: "merge aprovado"
      saida: "release reproduzível"
```

```

    executor: "esteira + humano"
    avanco: "build imutavel e deploy estagiado"
-   id: operar
    entrada: "release em producao"
    saida: "metricas + logs + painel"
    executor: "observabilidade"
    avanco: "contrato de saude respeitado"
-   id: evoluir
    entrada: "observacoes + erros"
    saida: "skills + specs revisadas + memoria"
    executor: "debriefing"
    avanco: "licoes executaveis capturadas"

```

O mapa é o contrato visível do ciclo: cada fase sabe o que recebe, o que entrega e o que precisa para avançar. É o primeiro artefato que o time versiona — e o alicerce de tudo o que vem a seguir.

O Estudo de Caso: Duas Organizações, Dois Destinos

Nada ilustra a tese do capítulo como o contraste entre duas organizações hipotéticas. A primeira, a **Organização Alfa**, adotou agentes como ferramenta de autocomplete: cada engenheiro usa a IA no IDE, o processo continua exatamente o mesmo, e as métricas de entrega melhoram discretamente — mas a taxa de bugs em produção também cresce, porque ninguém redesenhou o ciclo. A segunda, a **Organização Beta**, fez a transição completa: spec executável antes de todo build, verificação adversarial em três camadas, orçamento de contexto por fase e debriefing por ciclo. As duas usam os mesmos modelos de IA. Os resultados são incomparáveis.

A diferença não está na ferramenta — está no ciclo. A Alfa trocou o digitador, não o processo; a Beta redesenhou o processo inteiro em torno do novo executante. Esse contraste é o argumento central do livro: o SDLC AI-first não é sobre a IA, é sobre o contrato que a governa.

O Modelo de Decisão de Adoção

Antes de adotar o AI-first, o Comandante avalia a prontidão da organização — o modelo de decisão abaixo transforma a decisão de adoção em score:

```

def avaliar_prontidao(criterios: dict) -> dict:
    pesos = {"dados": 0.25, "processo": 0.25, "cultura": 0.20,
            "ferramentas": 0.15, "competencia": 0.15}
    score = sum(criterios.get(k, 0) * pesos[k] for k in pesos)
    veredito = ("pronta" if score >= 0.7 else
               "em_preparacao" if score >= 0.4 else "nao_pronta")
    return {"score": round(score, 2), "veredito": veredito}

```

```
PRONTIDAO = avaliar_prontidao({
    "dados": 0.8, "processo": 0.7, "cultura": 0.6,
    "ferramentas": 0.9, "competencia": 0.5,
})
print(f"Prontidao: {PRONTIDAO}")
```

O score de prontidão é o instrumento de decisão honesta: a organização sem competência de revisão adversarial (0.5) não está pronta para delegar sem supervisão — o modelo aponta o gap antes do investimento.

O Modelo de Gap de Fases com Custo de Contexto

O gap entre o ciclo tradicional e o AI-first não é uniforme — algumas fases ganham mais que outras. O modelo abaixo mede o ganho por fase em tempo e contexto, destacando onde a adoção rende mais:

```
def comparar_ciclos(fases, tempos_trad, tempos_ai, contexto_fase):
    resultado = []
    for i, fase in enumerate(fases):
        ganho = (tempos_trad[i] - tempos_ai[i]) / tempos_trad[i]
        resultado.append({'fase': fase, 'ganho_pct': round(ganho * 100, 1),
                        'contexto_fase': contexto_fase[i], 'prioridade':
                        'alta' if ganho > 0.5 else 'media' if ganho > 0.2 else 'baixa'})
    return sorted(resultado, key=lambda x: x['ganho_pct'], reverse=True)

fases = ['levantamento', 'spec', 'codigo', 'teste', 'revisao', 'deploy']
trad = [10, 8, 20, 6, 5, 2]
ai = [6, 3, 10, 4, 2, 1]
print(comparar_ciclos(fases, trad, ai, [0.4, 0.7, 0.9, 0.5, 0.8, 0.2]))
```

O ranking por ganho define onde começar a automação: a fase de código (50% de ganho) e a revisão (60%) são as primeiras candidatas, porque são as que mais consomem contexto e as que os agentes mais dominam hoje. O levantamento e o deploy, com ganho menor, ficam para a segunda onda — o que evita o erro de automatizar tudo de uma vez e não conseguir sustentar a operação.

O Modelo de Comparação entre Ciclos

A melhor forma de entender o AI-first é compará-lo com os modelos clássicos no mesmo quadro — mesmas dimensões, resultados diferentes. O quadro abaixo é o instrumento de comparação:

Dimensão	Waterfall	Ágil	AI-first
Artefato-mestre	Documento de requisitos	Backlog	Spec executável
Executante	Humano	Humano	Agente + humano
Verificação	Fase final	Final da iteração	Contínua e adversarial
Custo dominante	Horas	Horas	Tokens + contexto
Aprendizado	Post-mortem	Retrospectiva	Debriefing + skills
Fronteiras	Fases rígidas	Sprints	Contratos de fase
Escala de agentes	Não existe	Não existe	Lotes paralelos

A comparação não condena os modelos clássicos — eles resolveram o problema do seu tempo. O AI-first resolve o problema do seu tempo: executantes que não são humanos, e o custo de governá-los. O Comandante que domina os três modelos escolhe conscientemente — em vez de herdar o que a ferramenta impõe.

O Modelo de Avaliação de Prontidão do Time

O SDLC AI-first não se adota no vazio — o time precisa de prontidão técnica e cultural. O modelo abaixo aplica um questionário de prontidão e devolve um plano de desenvolvimento por lacuna:

```

PERGUNTAS_DE_PRONTIDIAO = [
    {'id': 'P1', 'dimensao': 'artefatos', 'pergunta': 'specs executaveis com criterio de aceite'},
    {'id': 'P2', 'dimensao': 'testes', 'pergunta': 'esteira de CI com testes automaticos'},
    {'id': 'P3', 'dimensao': 'contratos', 'pergunta': 'contratos de interface entre modulos'},
    {'id': 'P4', 'dimensao': 'cultura', 'pergunta': 'time disposto a revisar trabalho de agente'},
    {'id': 'P5', 'dimensao': 'metricas', 'pergunta': 'metricas de ciclo registradas ha 3 meses'},
]

def avaliar_prontidao(respostas):
    lacunas = []
    for p in PERGUNTAS_DE_PRONTIDIAO:
        if not respostas.get(p['id']):
            lacunas.append({'dimensao': p['dimensao'], 'acao': 'oficina_' + p['dimensao']})
    score = 1 - (len(lacunas) / len(PERGUNTAS_DE_PRONTIDIAO))

```

```
return {'score': round(score, 2), 'lacunas': lacunas,  
        'veredito': 'pronto' if score >= 0.8 else 'em_preparacao' if  
score >= 0.5 else 'nao_pronto'}  
  
print(avaliar_prontidao({'P1': True, 'P2': True, 'P3': False, 'P4': True,  
                        'P5': False}))
```

O veredito protege o investimento: um time que responde “não” para especificações e métricas não está pronto para delegar — os agentes vão produzir sobre areia e o fracasso será atribuído à IA, não ao processo. O plano de lacunas vira o primeiro backlog da adoção: oficina de artefatos, oficina de contratos, instalação de métricas. Só quando o score cruza 0.8 a primeira delegação agêntica é autorizada.

O Modelo de Análise de Gap do Ciclo

Todo campo nasce com vocabulário próprio — e o SDLC AI-first não é exceção. O glossário abaixo é o instrumento de comunicação do Comandante: os termos que o time inteiro usa com o mesmo significado.

Termo	Definição	Sinônimo proibido
Spec executável	Contrato de comportamento com requisitos e critérios de aceite	“documento de requisitos”
Verificação adversarial	Camada que tenta refutar o artefato com evidência	“revisão de código”
Evidência	Saída registrada de comando, diff ou métrica	“confio em você”
Orçamento de contexto	Tetos de tokens por fase, declarados e medidos	“limite do provedor”
Handoff	Documento de transferência de estado entre sessões	“perdi o contexto”
Worktree	Cópia isolada do repositório para um agente	“branch”
Caixa-preta	Registro estruturado de decisões do agente	“log de auditoria”
Debriefing	Extração de lições executáveis de incidentes	“post-mortem”
Radar	Sistema de verificação contínua e independente	“QA”
Torre de controle	Orquestração de fases com autoridade humana	“gerenciamento de projeto”

O glossário não é acadêmico — é operacional: cada termo proibido é um mal-entendido evitado entre humano, agente e verificação. O Comandante que adota o glossário elimina a classe mais cara de bug do AI-first: o bug de vocabulário.

O Modelo de Estimativa de Adoção

Antes de adotar o SDLC AI-first, a organização quer saber quanto vai custar. O modelo abaixo estima o esforço de adoção a partir de três variáveis: número de fases que serão agênticas, maturidade dos artefatos existentes e volume de código legado:

```
def estimar_adoção(fases_agentes, maturidade_artefatos, loc_legado):
    base = 40 # dias-homem de setup
    custo_fase = 6 * fases_agentes
    desconto_maturidade = maturidade_artefatos * 8
    custo_legado = loc_legado / 10000
    total = base + custo_fase - desconto_maturidade + custo_legado
    return {'total_dias': round(total, 1), 'fases_agentes': fases_agentes,
```

```
'legado': loc_legacy}

print(estimador_adocao(fases_agentes=8, maturidade_artefatos=3,
loc_legacy=120000))
# {'total_dias': 92.0, 'fases_agentes': 8, 'legado': 120000}
```

A fórmula é deliberadamente simples porque o objetivo não é precisão contábil — é criar uma conversa honesta sobre o custo de transição. Organizações que já têm artefatos maduros (specs executáveis, testes de contrato, esteiras de CI) pagam menos pela migração; organizações com monólito legado sem testes pagam mais. O número alimenta o cálculo de retorno da seção anterior: só faz sentido adotar se o ganho de produtividade projetado supera o custo de adoção em menos de um ano.

O Modelo de Análise de Gap do Ciclo

A transição entre o ciclo atual e o alvo AI-first exige um modelo de análise de gap — a lista do que existe, do que falta e do que sobra. O instrumento abaixo é o formato de análise:

```
analise_gap:
  organizacao: "Beta"
  fase_atual: "ciclo_classico_com_autocomplete"
  lacunas:
    - "sem spec executavel; artefato-mestre e o ticket"
    - "verificacao so no fim; sem camada adversarial"
    - "custo de contexto invisivel; sem orcamento"
    - "sem debriefing estruturado; licoes perdidas"
  sobras:
    - "praticas de revisao de codigo existentes"
    - "suites de teste legadas aproveitaveis"
    - "cultura de documentacao razoavel"
  plano_transicao:
    - "mes 1: espec executavel no fluxo piloto"
    - "mes 2: verificacao adversarial no CI"
    - "mes 3: orcamento de contexto por fase"
    - "mes 4: debriefing e banco de licoes"
```

A análise de gap é o mapa da transição: lacunas viram trabalho, sobras viram alavancas, e o plano vira o cronograma — a diferença entre mudar de processo e torcer para mudar.

O Monitor de Fases em Tempo Real

O contrato de fase só tem valor se alguém monitora o cumprimento em tempo real. O monitor abaixo acompanha cada fase da esteira, registra o desvio de caracteres em relação ao orçamento e aborta a transição quando o artefato de saída não existe:

```

import json
from pathlib import Path
from datetime import datetime

class MonitorDeFases:
    def __init__(self, contrato):
        self.contrato = contrato
        self.estado = {}

    def registrar_entrada(self, fase, artefato):
        if not Path(artefato).exists():
            raise RuntimeError(f'fase {fase} iniciada sem artefato de
entrada: {artefato}')
        self.estado[fase] = {'entrada': artefato, 'inicio':
datetime.now().isoformat()}

    def registrar_saida(self, fase, artefato, caracteres_previstos):
        if not Path(artefato).exists():
            raise RuntimeError(f'fase {fase} nao produziu artefato de
saida: {artefato}')
        tamanho = Path(artefato).stat().st_size
        desvio = tamanho - caracteres_previstos
        self.estado[fase]['saida'] = artefato
        self.estado[fase]['desvio'] = desvio
        self.estado[fase]['fim'] = datetime.now().isoformat()
        return desvio

    def relatorio(self):
        return json.dumps(self.estado, indent=2, ensure_ascii=False)

monitor = MonitorDeFases(contrato='contrato_fases.json')
monitor.registrar_entrada('planejamento', 'dossie.md')
monitor.registrar_saida('planejamento', 'sumario_macro.json', 8000)
print(monitor.relatorio())

```

O monitor transforma o contrato em disciplina: nenhuma fase abre sem artefato de entrada e nenhuma fecha sem artefato de saída. Quando o desvio de tamanho ultrapassa um limiar configurável (por exemplo, 20% para mais ou para menos), o monitor emite um alerta no relatório — é o sinal de que o agente está inventando escopo ou cortando conteúdo sem autorização. Esse dado alimenta a auditoria determinística que roda antes da compilação, e vira evidência no debriefing da fase.

O Modelo de Gate com Critérios Pesados

Nem todo critério de gate tem o mesmo peso. O modelo abaixo atribui pesos aos critérios de transição e calcula uma nota ponderada — um artefato pode ser aprovado mesmo com uma falha menor, desde que os critérios críticos estejam satisfeitos:

```
CRITERIOS = {
    'existencia': {'peso': 30, 'critico': True},
    'tamanho_minimo': {'peso': 25, 'critico': True},
    'referencias': {'peso': 20, 'critico': False},
    'diagrama': {'peso': 15, 'critico': False},
    'codigo_valida': {'peso': 10, 'critico': True},
}

def avaliar_gate(resultados):
    nota = 0.0
    criticos_falhos = []
    for criterio, config in CRITERIOS.items():
        ok = resultados.get(criterio, False)
        if ok:
            nota += config['peso']
        elif config['critico']:
            criticos_falhos.append(criterio)
    aprovado = nota >= 80 and not criticos_falhos
    return {'aprovado': aprovado, 'nota': nota, 'criticos_falhos':
criticos_falhos}

print(avaliar_gate({'existencia': True, 'tamanho_minimo': True,
'referencias': False, 'diagrama': True, 'codigo_valida': True}))
# {'aprovado': True, 'nota': 80.0, 'criticos_falhos': []}
```

A ponderação é uma decisão de governança: a organização decide quais critérios são inegociáveis (existência, tamanho, código válido) e quais admitem débito temporário (número de referências, presença de diagrama). O mesmo modelo roda em todas as fases, mas com pesos diferentes — na fase de redação as referências pesam mais, na fase de compilação a validade do código é tudo. Registrar a matriz de pesos como dado permite comparar rigor entre projetos e calibrar os limiares com base em evidência histórica, não em opinião.

A Esteira de Validação de Transição

A transição entre fases é o ponto onde o AI-first difere do clássico: ela é validada por esteira, não por reunião. O script abaixo é o gate de transição que impede uma fase de avançar sem o artefato de saída:

```

import json
from pathlib import Path
from typing import Dict

CONTRATO_FASES = {
    "spec": {"arquivo": "spec_executavel.json", "tipo": "json"},
    "design": {"arquivo": "contratos/modulos.json", "tipo": "json"},
    "build": {"arquivo": "resultado_ci.txt", "tipo": "texto"},
    "verificar": {"arquivo": "parecer_adversarial.md", "tipo": "texto"},
}

def validar_transicao(fase: str, raiz: Path) -> None:
    contrato = CONTRATO_FASES.get(fase)
    if contrato is None:
        raise ValueError(f"fase desconhecida: {fase}")
    caminho = raiz / contrato["arquivo"]
    if not caminho.exists():
        raise RuntimeError(f"transicao bloqueada: {contrato['arquivo']}
inexistente")
    if contrato["tipo"] == "json":
        json.loads(caminho.read_text(encoding="utf-8"))
    print(f"[OK] transicao {fase} validada com artefato {caminho.name}")

if __name__ == "__main__":
    validar_transicao("spec", Path("."))

```

A esteira de transição é a diferença entre “combinamos” e “é verificado”: nenhuma fase avança por confiança — avança por artefato validado.

O Modelo de Registro de Fase com Evidência

Cada fase encerrada precisa deixar evidência — senão o ciclo vira conversa. O modelo abaixo registra a conclusão de fase com o artefato de saída e o responsável:

```

class RegistroDeFase:
    def __init__(self):
        self.fases = []

    def concluir(self, fase, artefato, responsavel, aprovado):
        registro = {'fase': fase, 'artefato': artefato, 'responsavel':
responsavel, 'aprovado': aprovado}
        self.fases.append(registro)
        return registro

```

```
def pendentes(self):
    return [f for f in self.fases if not f['aprovado']]

r = RegistroDeFase()
r.concluir('esboco', 'sumario_macro.json', 'arquiteto', True)
r.concluir('redacao', 'cap_01.md', 'escritor', False)
print(r.pendentes())
```

O registro por fase cria a trilha auditável do ciclo inteiro: quem fez o quê, com qual artefato e se foi aprovado. Quando o capítulo 1 volta sem aprovação, o `pendentes()` mostra a dívida antes de o ciclo avançar — nada de empurrar fase não aprovada para frente e descobrir o problema no final.

O Diagnóstico do Ciclo Atual em Cinco Perguntas

Antes de redesenhar o ciclo, o Comandante diagnostica o estado atual. As cinco perguntas abaixo são o instrumento de diagnóstico — e a resposta para cada uma define o ponto de partida da mudança:

1. **Quem executa hoje?** Se a resposta é “sempre humano”, você está no SDLC clássico — e a mudança começa pela delegação de tarefas mecânicas.
2. **Qual é o artefato-mestre?** Se é o ticket ou o documento, a transição para spec executável será o primeiro salto estrutural.
3. **Quando a verificação acontece?** Se é só no fim, a verificação adversarial contínua será a segunda mudança.
4. **Qual é o recurso escasso?** Se você não sabe a resposta, o custo dominante ainda é invisível para o time.
5. **Onde está a evidência das decisões?** Se não existe registro consultável, o aprendizado do ciclo está sendo perdido.

O diagnóstico é o mapa antes do voo: sem ele, a mudança é uma aposta; com ele, a mudança é um plano.

O Exemplo do Mundo Real: a Fábrica Agêntica

Um caso concreto ajuda a ancorar o modelo. A Fábrica Agêntica de Livros é um pipeline editorial que produz obras técnicas completas do tema ao PDF ABNT — e exemplifica o SDLC AI-first na prática. O pipeline roda fases estanques com artefatos de transição obrigatórios: pesquisa gera o dossiê, arquitetura gera o sumário macro, redatores produzem capítulos EITA-V2 em lotes de quatro, um auditor determinístico valida requisitos contratuais, e um revisor técnico independente corrige os defeitos apontados.

O paralelo com o ciclo de software é direto: a spec executável é o sumário macro com requisitos; a verificação adversarial é o auditor determinístico que refuta capítulos fora do padrão; o debriefing é a revisão que alimenta a próxima rodada. O caso mostra que o modelo não é teórico — é operacional, com artefatos, métricas e gates.

A Economia da Mudança: o Cálculo do Retorno

A adoção do AI-first tem custo de transição — e o Comandante precisa do cálculo antes de vender a mudança para a organização. O modelo de retorno abaixo estima o ponto de equilíbrio:

```
def calcular_retorno(delegavel_pct: float, custo_hora_humana: float,
                    custo_tokens_por_hora: float, economia_velocidade:
float) -> float:
    """Retorno mensal estimado da delegacao agêntica."""
    horas_por_mes = 160.0
    horas_delegaveis = horas_por_mes * delegavel_pct
    custo_antes = horas_delegaveis * custo_hora_humana
    custo_depois = horas_delegaveis * custo_tokens_por_hora /
economia_velocidade
    return round(custo_antes - custo_depois, 2)

retorno = calcular_retorno(delegavel_pct=0.35,
                           custo_hora_humana=180.0,
                           custo_tokens_por_hora=60.0,
                           economia_velocidade=3.0)
print(f"Economia mensal estimada: R$ {retorno}")
```

O modelo é simplificado, mas captura a essência: a delegação só compensa quando o custo de tokens é conhecido — e é esse conhecimento que o capítulo inteiro constrói.

O Manifesto do Ciclo

O capítulo termina com o manifesto que a equipe adota — cinco compromissos curtos: toda fase decola com artefato de entrada aprovado e aterra com artefato de saída verificado; toda delegação tem contrato, evidência e reversibilidade; todo defeito detectado cedo é vitória, não vergonha; o custo de contexto é medido e orçado como qualquer recurso; e a responsabilidade final nunca é delegada, apenas a execução. O manifesto não é poesia — é a lista de verificação do contrato de fases traduzida em linguagem humana. Colado na parede do time, ele faz o papel do contrato quando a esteira ainda não existe.

Passos Numerados para Adotar o Modelo na Sua Equipe

1. **Inventarie as fases atuais** do seu ciclo (mesmo que informal). Liste o artefato de saída de cada fase.
2. **Classifique cada fase** como humana (decisão), agêntica (execução) ou mista.
3. **Defina o artefato obrigatório** de cada fase e o critério de validação.
4. **Programe a autorização de avanço** com o motor RACI do exemplo acima.
5. **Instrumente o consumo de tokens** por fase.
6. **Rode um piloto** em uma feature pequena antes de migrar a equipe inteira.

Aplica

Vamos colocar você, Comandante de Operações de Software, dentro de uma cena real. Você trabalha em uma scale-up de pagamentos com 40 engenheiros. O CTO voltou de uma conferência empolgado com agentes de IA e decretou: “quero que todo mundo use o agente para escrever código ainda este mês”. Os engenheiros adotam a ferramenta com entusiasmo — e, em seis semanas, a frequência de deploy sobe 60%. Parece ótimo até o primeiro mês de produção: a taxa de mudanças que causam falha dobra, um incidente de cobrança duplicada vaza para o cliente, e o time de QA — que não foi ampliado — afunda.

O erro não foi adotar agentes. O erro foi **adotar agentes sem redesenhar o ciclo de vida**. A equipe manteve o processo clássico (humano escreve, humano revisa, QA testa no fim) e apenas trocou o digitador por um agente. O throughput subiu; a estabilidade despencou — exatamente o padrão que o DORA 2025 documenta quando a governança não acompanha a adoção de IA.

O diagnóstico, ligado à teoria do capítulo: o artefato-mestre continuou sendo o ticket de Jira (um documento), não a spec executável. Nenhuma fase tinha contrato de saída validável por máquina. O agente decolava sem plano de voo, e o radar (observabilidade) não cobria o comportamento do próprio agente.

A correção prática, que você aplica na segunda-feira:

1. **Congele novas features por 2 semanas** e invista em definir, para cada fluxo, o artefato obrigatório de entrada e saída.
2. **Exija spec executável** para qualquer trabalho delegado a agente: escopo, critérios de aceite, casos de borda. Sem spec, sem build — a decolagem é bloqueada no código.
3. **Separe a verificação do build**: o agente que escreveu não valida sozinho; um segundo agente (ou revisor humano) tenta refutar.
4. **Meça tokens por feature** e aloque um orçamento de contexto, como você alocaria um orçamento de horas.

5. **Reuna o time semanalmente** para revisar não apenas o código, mas o contrato do ciclo: onde o processo travou, onde o agente avançou sem autorização, qual fase produziu mais retrabalho.

Armadilhas comuns que você verá no campo: tratar prompt engineering como se fosse governance (não é — é só comunicação); criar specs decorativas que ninguém valida; deixar o agente operar diretamente no working dir principal sem isolamento; e esquecer que o custo do contexto aparece no fim do mês, não no fim da feature.

Conclusão

Você fechou o primeiro voo com três marcos. Primeiro: entendeu que o SDLC clássico otimiza fases manuais e assume o humano como executante, enquanto o AI-first assume o agente como executante e o humano como orquestrador e árbitro. Segundo: viu que o artefato-mestre mudou — de documento para spec executável mais testes — e que isso é uma decisão de engenharia programável, demonstrada no motor de autorização de fases. Terceiro: reconheceu que o custo dominante mudou de horas-homem para tokens e contexto, o que exige instrumentação e governança próprias.

Como desafio, faça o inventário do ciclo de vida da sua equipe atual: liste as fases, os artefatos de saída e quem autoriza cada transição. Você vai descobrir que a maioria das “fases” não tem artefato — só reunião.

No próximo capítulo, você vai assumir o posto de controlador de voo de fato: a matriz de papéis humana, agêntica e de verificação que distribui responsabilidade em cada fase do ciclo.

Por que este capítulo importa

Se você chegou até aqui, já percebeu que do sdlc clássico ao ai-first: por que o contrato mudou. Este capítulo — *Capítulo 1: Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou* — é um convite para parar de usar IA como ferramenta de autocomplete e começar a tratá-la como parte estrutural do seu processo de desenvolvimento. A diferença entre as duas posturas é exatamente o que separa quem apenas acelera o que já fazia de quem repensa o que é possível.

Vamos explorar isso com exemplos práticos, código real e um passo a passo que você pode aplicar ainda hoje, sem esperar por infraestrutura nova ou aprovação de comitê. A ideia é simples: cada seção termina com algo que você pode executar em menos de uma hora.

Conceitos-chave deste capítulo

- **O contrato antes da execução:** antes de deixar qualquer agente trabalhar, defina o que significa “feito” em termos verificáveis.
- **Evidência antes de afirmação:** o que não pode ser verificado não pode ser delegado com segurança.
- **Aprendizado contínuo:** cada entrega, boa ou ruim, é matéria-prima para o próximo ciclo.

Esses três conceitos aparecem, de formas diferentes, em todas as seções a seguir. Mantê-los em mente enquanto você lê vai transformar exemplos isolados em um padrão que você reconhece na sua própria rotina.

Checklist para aplicar hoje

1. Escolha uma tarefa pequena e bem definida do seu backlog.
2. Escreva o critério de aceite em uma frase verificável.
3. Delegue a execução a um agente, mantendo a verificação em suas mãos.
4. Registre o que funcionou e o que não funcionou.
5. Transforme o aprendizado em um procedimento reutilizável.

Se você fizer apenas o primeiro item, já estará à frente da maioria das equipes — que continua discutindo IA em reuniões sem nunca definir o que quer que ela faça.

Perguntas que você deve se fazer

1. Qual fase do meu processo consome mais tempo hoje — e por quê?
2. O que eu delegaria a um agente amanhã se tivesse certeza de que o resultado seria verificado?
3. Qual informação eu poderia registrar hoje que tornaria a próxima iteração mais barata?
4. Quem no meu time revisa o trabalho de quem — e com qual critério?
5. O que eu faria se o custo de cada tentativa caísse para quase zero?

Essas perguntas não têm resposta certa, mas têm uma propriedade em comum: elas forçam você a sair da conversa abstrata sobre IA e entrar no terreno do seu processo real. E é exatamente nesse terreno que o SDLC AI-first produz resultado.

Glossário rápido

- **Agente:** programa que usa um modelo de linguagem para planejar e executar tarefas com acesso a ferramentas.
- **Harness:** a camada que conecta o agente ao ambiente — arquivos, comandos, testes e regras.
- **Spec executável:** especificação cujos critérios podem ser verificados por máquina.
- **Verificação adversarial:** camada que refuta o trabalho produzido, em vez de apenas confirmá-lo.
- **Contexto:** a janela de informação que o modelo enxerga a cada passo — o recurso mais caro do ciclo.

Dominar esses cinco termos é suficiente para acompanhar qualquer discussão séria sobre desenvolvimento orientado a agentes.

O erro mais comum nesta fase

A maioria das equipes comete o mesmo erro: adota a ferramenta e mantém o processo. O agente entra no fluxo como um autocomplete sofisticado, e todo o potencial de transformação se perde em pequenas conveniências. O antídoto é simples e desconfortável: mude o processo primeiro, depois traga a ferramenta. Defina o contrato, o critério de aceite e a verificação antes de permitir que o agente produza em escala. É contra intuitivo, mas é o que separa as equipes que capturam valor das que apenas geram volume.

Um exemplo concreto para fixar

Imagine uma pequena feature de faturamento. No fluxo tradicional, um desenvolvedor recebe a tarefa, interpreta a intenção, escreve o código e um revisor confia na leitura. No fluxo orientado a agentes, a mesma feature começa com uma frase verificável: “o valor total deve considerar o desconto aplicado antes dos impostos”. O agente implementa, os testes verificam a regra, e o humano revisa a evidência — não o código linha a linha, mas o comportamento observado. Perceba o deslocamento: o humano deixa de ler tudo para auditar o essencial, e o agente deixa de adivinhar para executar contra um critério. É essa troca que o restante do livro explora em profundidade.

Próximos Passos

Você acabou de percorrer um dos capítulos centrais do SDLC AI-first. Se este conteúdo fez sentido para você, o próximo passo natural é continuar a jornada pelo livro completo, que aprofunda cada fase do ciclo: especificação executável, design de domínio, harness e agentes, verificação adversarial, entrega segura, aprendizado contínuo, economia de tokens e governança de maturidade.

Enquanto isso, aqui vão três ações concretas:

1. **Pratique em um projeto pequeno.** Nada substitui experimentar com as próprias mãos — escolha um repositório pessoal e aplique um dos conceitos deste capítulo.
2. **Formalize seu contrato.** Escreva o critério de aceite de uma tarefa real antes de delegá-la. É um exercício de cinco minutos que muda completamente a qualidade do resultado.
3. **Mensure seu processo.** Registre quanto tempo e quanto contexto cada fase consome. O que não é medido não pode ser melhorado.

O céu do software agora tem torres de controle. O próximo voo é seu.

Boa leitura e bons voos.

— Heverton Eduardo Peres.

